实验 4：程序规约与定理证明

1. 实验主题

使用 ACSL 与 Frama-C 为 C 程序编写并验证规约，再使用 Rocq 完成小型、可由机器检查的交互式证明。

2. 实验目标

- 使用 ANSI/ISO C 规约语言（ANSI/ISO C Specification Language，ACSL）编写前置条件、后置条件、内存有效性条件和 `assigns` 子句。
- 理解 Frama-C 的最弱前置条件插件（Weakest Precondition，WP）产生的证明义务、Eva 值分析插件的抽象解释结果以及二者的差异。
- 逐步设计循环不变式（loop invariant），并识别过强前置条件和弱后置条件造成的空洞证明（vacuous proof）。
- 使用 Rocq 完成一个小型交互证明，并解释中间证明状态。

允许使用 LLM 生成候选规约、循环不变式或证明步骤，但必须通过实际工具运行、反例或规约变异核验。如果使用 LLM 生成正式规约、证明脚本或程序，实验报告还应包含实际提示词、你最终采用或修改的内容和独立核验结果。

3. 工具与参考资料

- Frama-C Documentation
- Frama-C Tutorials
- ACSL 语言参考
- The Rocq Prover
- Rocq 官方安装与编辑器配置说明

按照官方说明自行安装 Rocq 及其标准库，并配置支持交互证明的编辑器，例如 VS Code 配合 VsRocq 扩展，或 RocqIDE。所用编辑器应能够显示当前证明目标和已有假设，以完成任务 6 中的证明状态观察。

4. 最小环境检查（Smoke Test）

4.1 Frama-C/Eva 与 WP

保存以下文件为 `inc.c`：

```
/*@
  requires x < 2147483647;
  assigns \nothing;
  ensures \result == x + 1;
*/
int inc(int x) {
    return x + 1;
}

int main(void) {
```

```
    return inc(1) != 2;
}
```

运行：

```
frama-c -eva inc.c
frama-c -wp -wp-rte inc.c
```

环境正确时，Eva 不应报告由该程序产生的运行时告警；WP 应生成证明义务，并把简单算术性质证明为有效。若外部证明器未配置，应至少看到由内置化简器完成的目标和明确的未证明目标，而不是插件缺失错误。

4.2 Rocq

保存以下文件为 `rocq_smoke.v`：

```
From Stdlib Require Import List Arith Lia.

Theorem plus_zero : forall n : nat, n + 0 = n.
Proof.
  intros n.
  induction n; simpl; auto.
Qed.
```

在已配置的终端中查看版本并编译文件：

```
rocq -v
rocq compile rocq_smoke.v
```

环境正确时，版本信息正常显示，文件编译通过，且没有标准库加载失败、未完成证明或语法错误。随后在所配置的编辑器中打开同一文件，逐步执行证明，确认能够查看当前目标和假设。

5. 实验任务

任务 1：为区间截断函数编写并检查契约

任务摘要： 从自然语言要求中提取前置条件和分情况后置条件，为下面给出的 C 函数编写 ACSL 契约并检查契约能否发现实现错误。

保存以下实现为 `clamp_value.c`：

```
int clamp_value(int x, int low, int high) {
    if (x < low) {
        return low;
    }
    if (x > high) {
```

```
        return high;
    }
    return x;
}
```

自然语言要求如下：

- 调用时保证 `low <= high`。
- 当 `x < low` 时返回 `low`；当 `x > high` 时返回 `high`；否则返回 `x`。

完成以下工作：

1. 建立一张“输入区域—预期返回值”表，区分 `x < low`、`low <= x <= high` 和 `x > high`，并说明两个端点属于哪个区域。
2. 编写包含 `requires`、`assigns` 和分情况 `ensures` 的 ACSL 契约，使其准确表达上述要求；不得通过增加与需求无关的前置条件排除某一输入区域。
3. 使用 `frama-c -wp -wp-rte clamp_value.c` 验证，并把每一组后置条件对应到程序中的一个返回分支。
4. 复制一份错误版本，把 `x > high` 分支的返回值改为 `low`。使用同一契约检查错误版本，指出失败的证明义务对应哪条自然语言要求。

实验报告应包含： 输入区域表、完整 ACSL 契约、正确版本和错误版本的 WP 结果，以及失败证明义务与需求的对应；正确与错误版本关键证明结果的截图；以及你对契约如何区分正确实现和错误实现自行撰写的结论。

任务 2：内存有效性与副作用边界

任务摘要： 为固定的数组写入函数补充下标范围、内存有效性和副作用约束，说明每条契约为什么是合法访问所必需的。

保存以下程序为 `write_at.c`：

```
#include <stddef.h>

/*@
  ensures a[index] == value;
*/
void write_at(int *a, size_t n, size_t index, int value) {
    a[index] = value;
}
```

1. 保持初始契约不变，运行 `frama-c -wp -wp-rte write_at.c`，记录与 `a[index]` 的合法读写有关的未证明目标，并说明仅有功能后置条件为什么不足以保证内存安全。
2. 补充 `index < n`、数组区间 `a + (0 .. n-1)` 可写和 `assigns a[index]`，重新验证。逐条说明这些条件分别限制了什么调用或副作用。
3. 把整段数组有效性条件替换为只要求 `a + index` 指向一个可写元素，再次验证。比较两种写法：哪一种更准确地描述本函数实际访问的内存，哪一种向调用者要求了更完整的数组区域。
4. 分别删除 `index < n` 和 `assigns a[index]` 后重新检查，记录证明结果或契约含义的变化；恢复最终契约。

实验报告应包含： 初始代码、最终 ACSL 契约、补充前后的证明结果、整段数组与单个元素两种有效性条件的比较，以及删除范围或副作用约束后的结果；关键证明义务的截图；以及你对内存有效性条件和 `assigns` 分别说明什么自行撰写的结论。

任务 3：循环不变式与变式

任务摘要： 逐步为循环补充索引范围、计数范围和终止度量，观察每一条信息对应哪些证明义务。

保存以下程序为 `count_nonzero.c`：

```
#include <limits.h>
#include <stddef.h>

/*@
  requires n == 0 || \valid_read(a + (0 .. n-1));
  requires n <= INT_MAX;
  assigns \nothing;
  ensures 0 <= \result <= n;
*/
int count_nonzero(const int *a, size_t n) {
    size_t i = 0;
    int count = 0;
    while (i < n) {
        if (a[i] != 0) {
            ++count;
        }
        ++i;
    }
    return count;
}
```

1. 说明前置条件 `n <= INT_MAX` 为什么使 `int count` 和 `int` 返回值能够容纳最大可能计数；再为 `count_nonzero.c` 依次添加 `0 <= i <= n`、`0 <= count <= i` 和 `loop assigns i, count`。说明“已处理元素数为 `i`”为什么能够支持 `count <= i`。
2. 每次只添加上一步中的一项并重新运行 WP，记录它帮助证明了哪个范围或退出目标，不得直接粘贴最终版本。
3. 添加 `loop variant` 证明终止。然后删除或削弱一条关键不变式，观察哪个证明目标再次失败，并解释该不变式提供了什么信息。

实验报告应包含： `n <= INT_MAX` 与计数类型的关系、三项循环注解逐步添加的记录、最终不变式与 `loop variant`、删除一条关键不变式后的失败结果；关键失败与成功证明状态的截图；以及你对输入范围、索引范围、计数范围和终止度量各自作用自行撰写的结论。

任务 4：Eva 告警与数值边界

任务摘要： 在固定数组累加程序上依次缩小长度和值域，观察 Eva 的内存与溢出告警如何变化，再与 WP 对同一边界性质的证明进行比较。

保存以下程序为 `eva_sum.c`：

```c
#include "__fc_builtin.h"

int sum_prefix(const int *a, unsigned n) {
    int sum = 0;
    for (unsigned i = 0; i < n; ++i) {
        sum += a[i];
    }
    return sum;
}

int main(void) {
    int a[8];
    for (unsigned i = 0; i < 8; ++i) {
        a[i] = Frama_C_interval(-1000000000, 1000000000);
    }
    unsigned n = (unsigned)Frama_C_interval(0, 12);
    return sum_prefix(a, n);
}
```

1. 运行 `frama-c -eva -eva-slevel 20 eva_sum.c`，把告警分为数组访问、整数溢出和分析精度信息，并指出产生每类告警的输入范围。
2. 只把 `n` 的范围从 `0..12` 改为 `0..8`，重新运行并比较数组访问告警；其它部分保持不变。
3. 在长度已经限制为 `0..8` 的版本中，只把数组元素范围改为 `-100..100`，重新运行并比较整数溢出告警。说明最终结论依赖哪两个输入范围。
4. 为 `sum_prefix` 单独建立 `wp_sum.c`：用契约要求 `n <= 8`、访问区间可读且每个参与累加的元素位于 `-100..100`，用后置条件要求结果位于 `-800..800`；循环中使用 `0 <= i <= n` 和 `-100 * i <= sum <= 100 * i`。运行 `frama-c -wp -wp-rte wp_sum.c`，比较 Eva 在固定入口范围下不再告警与 WP 在契约前提下证明结果边界分别能说明什么。

实验报告应包含： 固定程序、三组 Eva 输入范围和告警对照、最终 `wp_sum.c` 契约与循环不变式、WP 结果，以及两种工具结论的前提；关键告警变化和证明结果的截图；以及你对“Eva 告警消失、WP 证明成功和业务需求正确为什么不是同一件事”自行撰写的结论。

任务 5：规约变异与非空洞性

任务摘要： 修改 `sign_of` 的规约并比较验证结果，理解“证明成功”为什么不一定表示规约完整表达了需求。

规约变异（specification mutation）是指每次只修改一处规约，再观察验证结果如何变化。

下面的 `sign_of` 函数在输入为负数、零和正数时，应分别返回 `-1`、`0` 和 `1`。将其保存为 `sign_of.c`：

```c
/*@
  assigns \nothing;
  ensures x < 0  ==> \result == -1;
  ensures x == 0 ==> \result == 0;
  ensures x > 0  ==> \result == 1;
*/
int sign_of(int x) {
    if (x < 0) {
```

```
        return -1;
    }
    if (x > 0) {
        return 1;
    }
    return 0;
}
```

以下三种变异互相独立。开始下一种变异前，恢复上面的程序和规约。每次运行后，都应把发生变化的证明义务对应到本次唯一修改的规约或程序位置。

1. **检查原始规约：** 运行 `frama-c -wp -wp-rte -wp-fct sign_of sign_of.c`，确认三条分情况后置条件均得到证明，并把每条成功的证明义务对应到负数、零或正数输入区域。
2. **削弱后置条件：** 将三条 `ensures` 替换为 `ensures -1 <= \result && \result <= 1;`，运行 WP。然后把函数改成始终返回 `0`，使用同一规约再次检查。解释错误实现为什么仍能通过，以及弱规约遗漏了什么要求。
3. **收紧前置条件：** 在原始规约中增加 `requires x != 0;`，检查 `sign_of`。然后在文件末尾增加 `int main(void) { return sign_of(0); }`，运行 `frama-c -wp -wp-rte -wp-fct main sign_of.c`。解释函数本身能够通过、零值调用却失败的原因。
4. **错写边界：** 将原始规约第一条后置条件中的 `x < 0` 改成 `x <= 0`，运行 WP。说明输入为 `0` 时，哪两条后置条件提出了不同要求。

实验报告应包含： 原始规约和三种变异、每次实际 WP 结果、发生变化的证明义务及其与修改位置的对应；原始规约成功、弱规约下错误实现仍成功、零值调用失败和错误边界失败的关键截图；以及你对“证明成功为什么不一定表示规约完整表达了需求”自行撰写的结论。

任务 6：Rocq 小型交互证明

任务摘要： 在 Rocq 中证明过滤后的列表长度不超过原列表长度，观察归纳证明状态，并比较显式证明与较短的自动化证明。

保存以下文件为 `filter_length.v`：

```
From Stdlib Require Import List Arith Lia.
Import ListNotations.

Theorem filter_length_le :
  forall (A : Type) (keep : A -> bool) (xs : list A),
    length (filter keep xs) <= length xs.
Proof.
  intros A keep xs.
  induction xs as [| x xs IH].
  - (* 空列表情形 *)
    admit.
  - (* 非空列表情形 *)
    simpl.
    destruct (keep x) eqn:Hkeep.
    + admit.
```

```
    + admit.
Qed.
```

1. 不使用 `Admitted` 或其它未完成证明占位，补全三个 `admit`，使文件能够通过 `rocq compile filter_length.v` 检查。
2. 在执行列表归纳后和对 `keep x` 分情况后，各保留一个中间证明状态，解释归纳假设、当前目标和下一步为什么可以使用算术自动化。
3. 在同一文件中以 `filter_length_le_short` 为名称写出同一结论的第二份证明，允许组合使用 `induction`、`simpl`、`destruct`、`auto` 或 `lia` 缩短脚本；比较两种脚本中结构是否清楚、自动化依赖是否明显。
4. 把结论临时改成严格不等式 `<`。在尝试修改证明脚本前，先判断新结论是否对所有列表成立；如果不成立，给出规模最小的具体输入并计算等式两侧的值。再重新编译，用该输入判断失败来自定理陈述还是证明策略；最后恢复正确结论。

实验报告应包含： 两份可检查的证明脚本、归纳后和分情况后的中间目标、两种策略对照，以及严格不等式版本的失败和你找到的最小反例；证明完成与中间目标的截图；以及你对如何区分“命题为假”和“证明策略不足”自行撰写的结论。

任务 7：验证 LLM 生成的小型 C 程序

任务摘要： 先独立写清一个小型 C 函数的需求，再让 LLM 生成实现，最后用你自己编写的 ACSL 契约和 Frama-C 检查该实现。

1. 从数值计算、区间映射、数组统计或边界检查中选择一个 10—25 行的 C 函数，先独立写出三条明确需求，其中至少一条涉及边界情况。
2. 把需求交给 LLM 生成第一版 C 程序，保留提示词、回答和未经人工修正的第一版代码。在首次验证前，不要为了让程序通过而修改需求。
3. 不依赖 LLM 的解释，自己为三条需求编写 ACSL 契约，并使用 Frama-C/WP 验证；如果程序包含数组、指针或可能溢出的运算，再使用 Eva 或运行时证明义务检查相应问题。
4. 如果验证失败，判断失败来自实现、规约还是缺少证明信息，修正一处后重新验证；如果验证成功，选择那条边界需求，指出哪个证明目标支持它。

实验报告应包含： 三条需求、提示词与 LLM 第一版代码、你独立编写的 ACSL 契约、Frama-C 结果，以及一次失败修正或一条成功证明义务的解释；关键运行结果的截图；以及你对 LLM 实现是否满足这三条需求自行撰写的结论。

6. 评分标准

本实验总分为 100 分。各项任务根据规约、证明脚本、运行结果、截图和分析评分。

任务	分值
任务 1：为区间截断函数编写并检查契约	20
任务 2：内存有效性与副作用边界	15
任务 3：循环不变式与变式	15
任务 4：Eva 告警与数值边界	15

任务	分值
任务 5：规约变异与非空洞性	10
任务 6：Rocq 小型交互证明	15
任务 7：验证 LLM 生成的小型 C 程序	10
总分	**100**